ECE 350: Real-Time Operating Systems Winter 2027

Lecture 5 — Access Control

Jeff Zarnett *2026-08-13*

Access Control

We established in the introduction that security and protection are important and cannot be left to the end of the course. We know why we need protection, and we have already had an introduction to the differences between policy and mechanism. Now, to focus on protection a bit further, we will take a deeper look at access control: who can do what in the system?

Principle of Least Privilege. One of the most impactful principles in the design of a system is the principle of least privilege. That is to say, every actor (whether a human, an agent, a program, etc.) should be given only the most limited set of privileges necessary to accomplish its goals. There are lots of obvious examples of why this makes sense: a finance team member should be able to look at the billing information in the cloud compute provider website – but not change any of the running instances. In an operating system, you would not permit a regular user to have the ability to change key operating system files, or those of other users. This is by no means a new idea: this was popularized and formalized in a paper from 1975 [SS75].

The benefit of such a system, of course, is that it minimizes the damage caused if something goes wrong, such as an errant or malicious user, a bug in some system, or a security compromise. We understand from learning about processes (and threads) already, that if there's an invalid memory access, the process is terminated with a Segmentation Fault, rather than permitting the access to proceed and change or access memory that belongs to another process (or the OS).

Despite that, being on the receiving end of the inevitable "no" answers that result from least-privilege rules is frustrating. You could legitimately ask: why can't I just `ssh` into the live server instance and read the log file to find out what went wrong? Do I have to ask an admin to help or get someone else to sign off? With unlimited permissions, it would be much easier and faster. The access control policy can be unpleasant to experience in this form, but – if I can `ssh` into the machine, what other things can I see?

Of course, it has recently become dramatically easier to motivate the principle of least privilege now that AI agents are widely used. AI agents do things quickly, and not only do they do things sometimes without asking, they do it when you *directly told them not to do it.* Consider this example from 2025: "An AI coding agent from Replit reportedly deleted a live database during a code freeze, prompting a response from the company's CEO. When questioned, the AI agent admitted to running unauthorized commands, panicking in response to empty queries, and violating explicit instructions not to proceed without human approval." [Nol25]. It is obvious, of course, that if the AI had simply not had the privileges to do that, this disaster likely could have been avoided. No further motivation is needed, then, agreed?

Discretionary Access Control (DAC)

Discretionary access control is used to refer to situations where the owner of something decides who may have access to it. That points us immediately towards talking about files, because those are the most salient example of things that users own and might want to grant permissions on. This is distinct from mandatory access control, which we'll discuss after this.

File Permissions. To protect users from one another and to maintain the confidentiality and integrity of data, files have associated permissions, which may control access to the following operations [SGG13]:

1. Read
2. Write
3. Execute
4. Append (write at the end of the file)
5. Delete
6. List (view the attributes of the file)

UNIX-Style Permissions. UNIX-Style permissions are commonly used still today in a lot of UNIX and UNIX-like systems. Each file has an owner and a group, and a set of permissions that can be assigned for the owner, the group, and for everyone. There are three basic permissions: read, write, and execute (run as a program). The permissions are represented using 10 bits, where a 1 indicates true and a 0 indicates false. The first bit is the directory bit and indicates if the file being examined is actually a directory. The next three bits are the read, write, and execute bits for the owner, followed by the read, write, and execute bits for the group, and finally the read, write, and execute bits for everyone.

Effective permissions are determined by the user: the owner of the file gets the owner permissions even if different permissions are assigned to the group or everyone. Precedence goes from left to right: owner takes precedence over the group; group takes precedence over the permissions for everyone.

The permissions can be shown to the screen in a human-readable format which is ten characters long. The order is always the same, and so a dash (`-`) appears if a bit is zero (permission does not exist). The character `d` is used to indicate a directory, `r` to indicate read access, `w` to indicate write access, and `x` to indicate execute access.

Example: permissions of `-rwxr-----` indicate that a file is not a directory; the owner can read, write, and execute; other members of the group can read it only, and everyone else has no access to the file (cannot read, write, or execute).

Permissions can also be written in octal (base 8) where r = 4, w = 2, and x = 1. To get the octal representation, start with 0, and then add the value of the permissions that are present, using zero where permissions are absent. You might then have a permission that reads `750` - meaning that the owner has read, write and execute access (0 + 4 + 2 + 1 = 7); group members have read and execute access (0 + 4 + 0 + 1 = 5); everyone else has no access to it at all.

`setuid` and `setgid`. These are advanced concepts and very easy to get wrong; in fact, there's a whole research paper from 2002 that argues that these are poorly designed, lack appropriate documentation, and are the causes of many security vulnerabilities [CWD02].

The thinking behind `setuid` is that it runs a program with the file owner's user ID rather than that of the person who invoked the program. The most basic example that is easy to motivate is the password change utility. A regular user should be able to change their password: good security practices involve rotating credentials when they are compromised or on a regular basis just in case. Yet, a regular user should not be able to read or write the password storage. The password change utility is owned by `root` and the invocation thereof by a regular user means we need to run with elevated permissions to accomplish the goal. `setuid` provides a pathway. The risk of this is that it's all-or-nothing: the power granted is not just that the application may access this file owned by `root` – it has the full powers of `root`.

This should immediately set off an alarm bell, because going all the way to `root` privileges is not at all the principle of least privilege. A better solution would instead give just the powers needed and no more. You may edit this password, but cannot delete important system files...

You may wonder if the existence of `sudo` removes the need for `setuid`: this allows me to do things with superuser permissions. Good question, but `sudo` uses the `setuid` mechanism to do what you asked. What `sudo`, a user-space program, brings to the party is the `sudoers` file: an administrator-written policy document that lays out

what users are permitted to do what in the system. The `sudo` command itself will refer to this policy document to decide whether to permit something to go ahead, but the mechanism remains `setuid`.

If `sudo` is not the solution, what is? The kernel's solution is called *capabilities*: decompose the privileges of `root` so that a program gets only what it needs to do the job. There are more than 30 of them listed in [Ker10] (and probably more since it was published). This does permit granting more specific and less-expansive privileges to a process when it runs. As an example, a process that gets the permission to use `CAP_NET_BIND_SERVICE` is permitted to bind a privileged port (remember, those are the ones below 1024); it's obviously preferable to grant *just* this power to an application program that is going to communicate over the network.

Of course, not everything an administrator can do has an associated capability, so there may not be a matching permission grant available. Another problem is that many powers have accumulated to the capability `CAP_SYS_ADMIN` as some sort of catch-all/fallback option. This has made it very similar to just having the full powers of `root`, so it represents a bit of a degradation of the model.

The `setgid` call is analogous to setting the user ID, except it sets the group ID instead. This can have the same goal: ensure that the execution of the process runs with the permissions of "someone else" rather than the invoker.

Most examples I could find motivating the use of `setuid` result in executing something as `root`, but in principle it could be as any user. The best example I could find for something using `setgid` is announcements. System administrators can write announcements (like "system is shutting down at 23:00 for maintenance") using the `wall` command – this uses the group ID necessary to write to everyone's terminal rather than `root`.

Access Control Lists. The obvious shortcoming of the UNIX permissions approach is that it is very coarse-grained: there are only three groups for whom we can specify permissions. Root users can also bypass everything, which may not be desirable. Within Linux and other similar systems there is a trend now towards using SELinux (Security Enhanced Linux) to put some limitations in. But first, let's talk about addressing that first limitation.

In NTFS (the Windows NT File System), files are protected by Access Control Lists. In such a system, each file can have as many security descriptors as we like, in which we specify the permissions a specific user or group is supposed to have. Similarly, the list of permissions need not be restricted to read-write-execute; NTFS, for example, has several different permissions such as "Take Ownership". This is still discretionary access control, just with more flexibility about what the permissions are and how they are assigned.

In NTFS, you can see the security descriptors for a file rather easily by right clicking it in Explorer, opening the properties dialog, and choosing the security tab. The descriptors have two checkboxes: allow and deny, with deny taking precedence over allow.

Permissions can start out at "default deny", in which case only the users explicitly granted access may access the file, or "default permit" in which case everyone has access, minus those users whose privileges are explicitly denied. Default deny is obviously more secure.

A simple way to represent an access control list is a set of tuples listing the users and permissions. For example:

```
(alice, read)
(bob, read)
(charlie, read)
(charlie, write)
(bob, execute)
```

Access Control Lists have a complexity that the UNIX file permissions do not: the idea of inheritance. We'll examine, briefly, inheritance in NTFS. Inheritance is supposed to be a convenience feature, but it can often be a cause of problems. If there is a directory with ACLs established and object inheritance is enabled, then any new file created in this directory will receive the same ACL of the directory. The danger is that any existing file moved into the directory will retain its original ACL. If it is supposed to get the ACL of the directory, it needs to be explicitly set [Dew04].

Mandatory Access Control (MAC)

Discretionary access control is a perfectly good and usable model, but is not perfect by any means. Mandatory access control policies are called that because they allow administrators to define a set of hard rules that cannot be overridden. This means that even if you own a file as a user, it does not mean you can do anything you want with it.

There are two major reasons for this: one is that we might want to stop users from doing things that hurt themselves or break rules. The second is putting restrictions on the `root` user permissions generally lessens the impact of any failure or compromise.

Users hurting themselves? Sure. Users copy-paste code they don't understand into scripts and run it and say "yes whatever shut up go away" to warning dialog boxes that tell them they should be careful. Or social engineering does the work of getting them to execute a command they should not run. So they can be tricked into running a program that does something bad. Or they install an update of a program they frequently use, and it has been compromised by a supply-chain attack. Or an AI agent can go off-track and do things you did not intend. There are clearly many pathways that lead to bad outcomes.

As for users breaking rules, nothing in discretionary access control stops them from doing that. Owning a file means you can copy it somewhere it is not meant to go. You had the permissions to do it, but you should not have. A lot of MAC literature also comes loaded with examples about secret and top-secret documents and ensuring that users cannot share things they have access to. Nothing in discretionary access control, for example, stops a user who has top-secret clearance from reading a document labelled top-secret and then copying that content to the `/tmp` directory where it can be seen by people who do not have clearance. They have the permission to read the file, and permission to write to `/tmp`. They are still going to prison in this scenario, but DAC could not stop this. MAC, however, can.

Restricting what `root` can do is also a goal of mandatory access control: it limits the damage that can be done if an attacker acquires elevated privileges. Red Hat gives an example involving a web server and a database server that co-exist on the same system, yet cannot access one another's files.

We are going to talk about one specific implementation of MAC in Linux: SELinux. This is not the only approach; there's a credible alternative called AppArmor used in Ubuntu Linux, but we will not talk about it for time reasons.

SELinux. SELinux brings in mandatory access controls as well as rules that go beyond the UNIX permissions scheme. The intention was to make something that is flexible enough to permit configuration, but still provides the mandatory access control guarantees needed. Let's go through a quick explanation from [Wal13].

The policy approach for SELinux is based on labelling: assign every process, file, network port, device, etc. a label. Processes are assigned types to identify what sort of process they are; to identify distinct processes of the same type, random categories are assigned. There also exists the concept of one level being contained within another (they call it dominance in SELinux).

The labels and types are then referenced in building up the rules. This goes alongside discretionary access control – Red Hat's documentation makes it clear that we check discretionary rules first, then mandatory ones. The policy administrators write the rules of what to allow; it is a deny-by-default sort of policy.

A major goal is restricting each process to the bare minimum of what it needs, to limit what it can do if compromised. Imagine a web server is compromised, but it cannot read files on the system outside of the web server directory, even those with permissions 777 because that permission is not granted in the MAC system.

The end goal here is that there is no all-powerful root account that is vulnerable to takeover or exploitation. That's difficult to achieve in practice, and can work only if enforcement is strict. But at least the system moves away from one account with all powers.

Suffice it to say that the enhanced security requires enforcement, which takes work, and therefore it slows down the execution of system calls. The original paper [LS01] shows the magnitude of these overhead costs, but the figures are over 25 years old and unlikely to still be a realistic representation of the modern costs.

We will not dip into the specifics of the implementation of enforcement of these policies, except to say that the kernel must check the policies when actions are taken. Much of the work of the implementation is, as you would expect, ensuring that all pathways check permissions with no possibility of an unintended bypass. That is too far in the implementation details for this course.

Cancel, or Allow? One of the most common experiences people have with SELinux is finding it to be too complicated to set up correctly, resulting in just... turning it off. This is counterproductive, for obvious reasons. Despite all the jokes we might make about the only secure computer being one that's turned off and encased in 2 metres of concrete, the concept of designing security systems so that people don't just turn them off because they're annoying is the subject of a graduate course I recommend: "ECE 750T38: Usable Security and Privacy".

To end that discussion less negatively, Android has been using SELinux (with it turned on, obviously!) for many years (according to the Android docs, since 5.0) and it has generally been a seamless experience for users. This is partly down to differing expectations of applications on mobile devices versus those on a server, but it is nevertheless a success story.

System Call Filtering & Sandboxing

As we know from the previous course, there are a great many things that an operating system offers to userspace programs through system call invocation. There are a tremendous number of system calls available, but many processes will never need most or all of them. In fact, it's dangerous to let a process have the ability to call these things even if we think it never will.

Let's make this more concrete: imagine this is a web server that serves pages. It should never delete files (this is maybe oversimplifying but go with it). If the process is simply unable to invoke the delete-file system call, then a compromise of the web-server means that it cannot delete files. At all.

The kernel docs[1] introduce a second motivation – there may be bugs in the kernel, and if a system call can be exploited then the attacker may gain total access to the system. If system calls we will never use are not available, then those exploit paths are blocked.

The implementation discussed in the kernel doc uses the Berkeley Packet Filter, which had its original usage as being for socket filtering. It's just turned inwards now, to restricting system calls: use the rules to filter system calls by their number and arguments. There's a limitation that it's only the value of the arguments, so if any of the arguments are pointers, the content is not available.

Landlock. The idea of restricting a system call entirely is important for things that we know will never be needed. If we go back to the web server deleting files example, though, we sometimes have legitimate use cases for a system call but want to apply some restrictions. Imagine that the web server should not be permitted to delete the content that it serves, but should be permitted to delete temporary files it uses for internal purposes. For that we need something more sophisticated. The concrete example we'll talk about is called *Landlock*[2].

Landlock's purpose is to let a program voluntarily put restrictions on itself and on child processes, and to get specific about what files, directories, and ports should be restricted. The authors of a user-space program would know well what should be permitted and what should not be, right?

The policy here is composed of rules in a default-deny environment – if we choose to include that type of action in consideration. Rules go in a ruleset and rules describe file system or network access rights. Then once the rules

[1] `https://docs.kernel.org/userspace-api/seccomp_filter.html`
[2] `https://man7.org/linux/man-pages/man7/landlock.7.html`

are written, apply them to yourself (and child processes). After that, the kernel enforces those rules as expected.

Let's take an example from Mikko Juola of a program that restricts its network usage to permit only incoming network connections and forbid outgoing ones: `https://github.com/Noeda/landlock-network-sandboxer-tool`. A simplified version of that with the relevant sections here:

```
attr.handled_access_net =
        LANDLOCK_ACCESS_NET_BIND_TCP |
        LANDLOCK_ACCESS_NET_CONNECT_TCP;

int ruleset_fd = landlock_create_ruleset(&attr, sizeof(attr), 0);
struct landlock_net_port_attr port_bind = {0};
port_bind.port = port;
port_bind.allowed_access = LANDLOCK_ACCESS_NET_BIND_TCP;
if (landlock_add_rule(ruleset_fd, LANDLOCK_RULE_NET_PORT, &port_bind, 0)) {
    perror("Failed to add a network rule with landlock_add_rule.\n");
    exit(1);
}

if (landlock_restrict_self(ruleset_fd, restrict_flags)) {
    perror("Failed to enforce landlock ruleset with landlock_restrict_self");
    exit(1);
}
close(ruleset_fd);
```

With this policy applied, we can be sure that we permit `bind()` on the one specific port using the rule about binding, but we forbid everything related to connect by having NO rule permitting the use of `connect()`.

The Landlock approach can, of course, be combined with others that we've seen. Putting them all together ensures that we have made sure to respect the principle of least privilege as best we can, and the benefits are returned to us by lower impact of security breaches or users (and their agents) gone rogue.

References

[CWD02] Hao Chen, David Wagner, and Drew Dean. Setuid demystified. In *11th USENIX Security Symposium (USENIX Security 02)*, San Francisco, CA, August 2002. USENIX Association. URL: `https://www.usenix.org/conference/11th-usenix-security-symposium/setuid-demystified`.

[Dew04] Brian Dewey. Understanding ACLs in NTFS, 2004. Online; accessed 13-December-2013. URL: `http://blogs.msdn.com/b/brian_dewey/archive/2004/01/20/60902.aspx`.

[Ker10] Michael Kerrisk. *The Linux Programming Interface*. No Starch Press Inc, 2010.

[LS01] Peter Loscocco and Stephen Smalley. Integrating flexible support for security policies into the linux operating system. In *2001 USENIX Annual Technical Conference (USENIX ATC 01)*, Boston, MA, June 2001. USENIX Association. URL: `https://www.usenix.org/conference/2001-usenix-annual-technical-conference/integrating-flexible-support-security-policies`.

[Nol25] Beatrice Nolan. An ai-powered coding tool wiped out a software company's database, then apologized for a 'catastrophic failure on my part', 2025. Online; accessed 12-August-2026. URL: `https://fortune.com/2025/07/23/ai-coding-tool-replit-wiped-database-called-it-a-catastrophic-failure/`.

[SGG13] Abraham Silberschatz, Peter Baer Galvin, and Greg Gagne. *Operating System Concepts (9th Edition)*. John Wiley & Sons, 2013.

[SS75] J.H. Saltzer and M.D. Schroeder. The protection of information in computer systems. *Proceedings of the IEEE*, 63(9):1278–1308, 1975. `doi:10.1109/PROC.1975.9939`.

[Wal13] Daniel J Walsh. Your visual how-to guide for selinux policy enforcement, 2013. Online; accessed 13-August-2026. URL: `https://opensource.com/business/13/11/selinux-policy-guide`.